ISA de 20 bits: Green Card, Descritivo e Convenção de Registradores

Gustavo Moro Conke (GRR20266205)

Nícolas Lazzarin Schneider (GRR20263677)

Parte 1: Green card

1.1 Características gerais

Item	Valor
Tamanho da instrução e da palavra	20 bits
Registradores	16 (x0 a x15), campos de 4 bits; x0 vale sempre 0
Arquitetura	load/store: somente lw e sw acessam a memória
Memória	endereçada por palavra (cada posição guarda 20 bits)
Contador de programa (PC)	20 bits, avança de 1 em 1 palavra
Opcode	4 bits, nos bits [19:16]
Imediatos	8 bits (formatos I, S e B) e 12 bits (formato J)
Microarquitetura	monociclo: uma instrução por ciclo de clock

1.2 Formatos de instrução

Formato	[19:16]	[15:12]	[11:8]	[7:4]	[3:0]
R	opcode	rd	rs1	rs2	funct
I	opcode	rd	rs1	imm[7:4]	imm[3:0]
S	opcode	rs2	rs1	imm[7:4]	imm[3:0]
B	opcode	rs2	rs1	imm[7:4]	imm[3:0]
J	opcode	rd	imm[11:8]	imm[7:4]	imm[3:0]

O campo rs1 fica sempre em [11:8]. Nos formatos S e B, o campo [15:12] guarda rs2 (registrador fonte), e não um destino.

1.3 Instruções tipo R (opcode 0000)

A operação é escolhida pelo campo funct.

Mnemônico	funct	Operação	Exemplo	Código (hex)
add rd, rs1, rs2	0000	rd = rs1 + rs2	add x6, x4, x5	06450
sub rd, rs1, rs2	0001	rd = rs1 − rs2	sub x6, x4, x5	06451
and rd, rs1, rs2	0010	rd = rs1 AND rs2 (bit a bit)	and x6, x4, x5	06452
or rd, rs1, rs2	0011	rd = rs1 OR rs2 (bit a bit)	or x6, x4, x5	06453
xor rd, rs1, rs2	0100	rd = rs1 XOR rs2 (bit a bit)	xor x6, x4, x5	06454
slt rd, rs1, rs2	0101	rd = 1 se rs1 < rs2 (com sinal), senão 0	slt x6, x4, x5	06455
sll rd, rs1, rs2	0110	rd = rs1 deslocado à esquerda por rs2[4:0]	sll x6, x4, x5	06456
srl rd, rs1, rs2	0111	rd = rs1 deslocado à direita (lógico) por rs2[4:0]	srl x6, x4, x5	06457
mul rd, rs1, rs2	1000	rd = 20 bits menos significativos de rs1 × rs2	mul x6, x4, x5	06458

1.4 Demais instruções

Mnemônico	Formato	Opcode	Operação	Imediato	Exemplo	Código (hex)
addi rd, rs1, imm	I	0001	rd = rs1 + imm	sinal (8)	addi x4, x0, 5	14005
andi rd, rs1, imm	I	0010	rd = rs1 AND imm	zeros (8)	andi x5, x4, 255	254FF
ori rd, rs1, imm	I	0011	rd = rs1 OR imm	zeros (8)	ori x5, x4, 3	35403
lw rd, imm(rs1)	S	0100	rd = MEM[rs1 + imm]	sinal (8)	lw x5, 2(x2)	45202
sw rs2, imm(rs1)	S	0101	MEM[rs1 + imm] = rs2	sinal (8)	sw x4, 2(x2)	54202
beq rs1, rs2, alvo	B	0110	se rs1 = rs2: PC = PC + 1 + imm	sinal (8)	beq x4, x5, -2	654FE
bne rs1, rs2, alvo	B	0111	se rs1 ≠ rs2: PC = PC + 1 + imm	sinal (8)	bne x4, x5, -2	754FE

Mnemônico	Formato	Opcode	Operação	Imediato	Exemplo	Código (hex)
blt rs1, rs2, alvo	B	1000	se rs1 < rs2 (com sinal): PC = PC + 1 + imm	sinal (8)	blt x4, x5, 3	85403
jal rd, alvo	J	1001	rd = PC + 1; PC = PC + 1 + imm	sinal (12)	jal x1, 2	91002
jalr rd, rs1, imm	J	1010	rd = PC + 1; PC = rs1 + imm	sinal (8)	jalr x0, x1, 0	A0100

Os opcodes 1011 a 1111 são reservados (não usados).

1.5 Desvios e saltos: semântica exata

Instrução	Condição	Próximo PC	Link (rd)	Alcance
beq	rs1 = rs2	PC + 1 + sext(imm8)	não há	−128 a +127 palavras
bne	rs1 ≠ rs2	PC + 1 + sext(imm8)	não há	−128 a +127 palavras
blt	rs1 < rs2, com sinal (complemento de dois)	PC + 1 + sext(imm8)	não há	−128 a +127 palavras
jal	sempre	PC + 1 + sext(imm12)	rd = PC + 1	−2048 a +2047 palavras
jalr	sempre	rs1 + sext(imm8), endereço absoluto	rd = PC + 1	qualquer endereço

Regras:

- Se a condição de um desvio é falsa, o próximo PC é PC + 1.
- O deslocamento é contado em palavras, a partir da instrução seguinte ao desvio (PC + 1), e não a partir da própria instrução.
- O montador calcula imm = endereço do alvo − (endereço da instrução + 1). Exemplo: um bne no endereço 3 que volta ao endereço 2 usa imm = 2 − 4 = −2 (0xFE), o que gera o código 754FE.
- PC + 1 é o endereço de retorno: jal e jalr gravam esse valor em rd. Se rd = x0, a gravação é descartada e a instrução vira um salto simples.
- No jalr, rs1 é lido antes de rd ser escrito, então jalr x1, x1, 0 é válido.
- Não existe delay slot: o desvio tem efeito no mesmo ciclo, e a instrução seguinte (PC + 1) só executa se o desvio não for tomado.
- A chamada de função é jal ra, função; o retorno é jalr zero, ra, 0 (ret).

1.6 Pseudoinstruções (resolvidas pelo montador)

Pseudo	Expande para
li rd, imm	addi rd, zero, imm
mv rd, rs	addi rd, rs, 0
nop	addi zero, zero, 0
j alvo	jal zero, alvo
call alvo	jal ra, alvo
ret	jalr zero, ra, 0

Parte 2: Descritivo da ISA

2.1 Visão geral

A ISA tem instruções de 20 bits e palavra de 20 bits: registradores, operandos, endereços calculados e posições de memória têm 20 bits. É uma arquitetura load/store com 16 registradores, em que a ALU opera somente sobre registradores. A memória de instruções e a memória de dados são separadas, ambas endereçadas por palavra, com 4096 posições de 20 bits cada.

2.2 Decisões de projeto e justificativas

Decisão	Justificativa
16 registradores (campos de 4 bits)	Os três campos do formato R (rd, rs1, rs2) ocupam 12 dos 20 bits, sobrando 4 para opcode e 4 para funct. Com 32 registradores sobraria só 1 bit de funct e os imediatos cairiam para 6 bits. Com 8 registradores faltariam registradores para argumentos, temporários e salvos. 16 registradores também é o número do RV32E, versão embarcada do RISC-V.
Opcode de 4 bits	16 opcodes bastam para as 11 classes de instrução; sobram 5 para extensões.
4 layouts distintos (R, I, S/B, J)	Cada layout atende uma necessidade: R para operações entre registradores, I para operações com imediato e carga, S/B para dois registradores fonte mais imediato, J para salto com um único registrador e imediato largo. S e B têm o mesmo layout e se diferenciam só pelo opcode; foram mantidos com nomes separados por clareza, como no RISC-V.

Decisão	Justificativa
rs1 sempre em [11:8]	O banco de registradores lê rs1 sem precisar de multiplexador.
Imediato de 8 bits (I, S, B)	Com dois campos de registrador na instrução, sobram 8 bits (−128 a +127), suficientes para constantes pequenas, deslocamentos de memória e desvios de if e laços.
Imediato de 12 bits (J)	O jal tem só o campo rd, então sobram 12 bits (−2048 a +2047), ampliando o alcance das chamadas de função.
andi e ori com extensão de zeros	Permite montar máscaras e constantes como 0xFF sem que o bit 7 vire uma sequência de 1s.
Memória endereçada por palavra	Cada posição guarda uma palavra de 20 bits, dispensando byte steering. O PC avança de 1 em 1.
x0 fixo em zero	Permite pseudoinstruções (li, mv, j, ret, nop) sem hardware extra.
Instrução mul	Usa o código funct 1000, livre no tipo R, e acelera a multiplicação de matrizes. Fica fora da ALU para não alterar o código de 3 bits da ALU nem a unidade de controle.

2.3 Campos da instrução

Campo	Bits	Largura	Uso
opcode	[19:16]	4	seleciona a instrução (ou o tipo R, quando vale 0000)
rd	[15:12]	4	registrador destino (R, I, J)
rs2	[7:4] no formato R; [15:12] nos formatos S e B	4	segundo registrador fonte
rs1	[11:8]	4	primeiro registrador fonte (R, I, S, B)
funct	[3:0]	4	operação do tipo R
imm8	[7:0]	8	imediato de I, S e B
imm12	[11:0]	12	imediato de J

2.4 Construção e extensão do imediato

Todo imediato é estendido para 20 bits antes de ser usado.

Instruções	Campo	Extensão	Como é feita
addi, lw, sw, beq, bne, blt, jalr	imm8 = I[7:0]	sinal	os bits 19 a 8 recebem uma cópia do bit 7
andi, ori	imm8 = I[7:0]	zeros	os bits 19 a 8 valem 0
jal	imm12 = I[11:0]	sinal	os bits 19 a 12 recebem uma cópia do bit 11

Exemplos:

- imm8 = 0xFF com extensão de sinal vale 0xFFFFF (−1); com extensão de zeros vale 0x000FF (255).
- imm8 = 0x7F com sinal vale 0x0007F (+127); imm8 = 0x80 com sinal vale 0xFFF80 (−128).
- imm12 = 0x800 com sinal vale 0xFF800 (−2048); imm12 = 0x7FF com sinal vale 0x007FF (+2047).

Faixas: sinal de 8 bits vai de −128 a +127; zeros de 8 bits vai de 0 a 255; sinal de 12 bits vai de −2048 a +2047.

Na implementação de referência, o gerador de imediato (ImmGen) recebe a instrução inteira (I) e um seletor de 2 bits (SEL): 00 = sinal de 8 bits, 01 = sinal de 12 bits, 11 = zeros de 8 bits. O valor 10 não é usado e repete o 00.

2.5 Registradores

São 16 registradores de 20 bits (x0 a x15). O x0 sempre lê 0 e qualquer escrita nele é ignorada. A tabela com nomes, funções e convenção de salvamento está na Parte 3.

2.6 Memória

- Memória de instruções (IMEM): 4096 palavras de 20 bits, endereçada pelos 12 bits menos significativos do PC.
- Memória de dados (DMEM): 4096 palavras de 20 bits, endereçada pelos 12 bits menos significativos do endereço efetivo (rs1 + imm).
- Endereçamento por palavra: o endereço n designa a n-ésima palavra, sem ordem de bytes e sem alinhamento.
- Só lw e sw acessam a DMEM. Os endereços calculados têm 20 bits; só os 12 bits baixos são usados no acesso, então endereços maiores que 4095 dão a volta.

2.7 Contador de programa e fluxo de controle

- O PC tem 20 bits e vale 0 após o reset.
- Em toda instrução, o próximo PC é PC + 1, exceto em desvios tomados, jal e jalr.
- O alvo de desvios e do jal é relativo a PC + 1; o alvo do jalr é absoluto (rs1 + imediato).
- Não existe instrução de parada. Por convenção, o fim de um programa é um desvio para si mesmo: fim: beq zero, zero, fim.

2.8 Semântica detalhada das operações

- Aritmética módulo 2^20: soma e subtração dão a volta em caso de estouro. Não há flags nem exceções.
- slt e blt comparam em complemento de dois (com sinal).
- sll e srl usam apenas rs2[4:0] como quantidade. São deslocamentos lógicos (entram zeros). Quantidades de 20 a 31 produzem 0.
- mul retorna os 20 bits menos significativos do produto. Esses bits são iguais para operandos com ou sem sinal.
- Instruções que escrevem em x0 não alteram o estado. lw e sw usam o endereço rs1 + sext(imm8).
- Opcodes 1011 a 1111 são reservados. Na implementação de referência executam como nop (nenhum registrador, memória ou PC além de PC + 1 é alterado).
- No tipo R, funct de 1001 a 1111 é reservado. Na implementação de referência, qualquer funct com o bit 3 em 1 executa mul.

2.9 Estado inicial e término

- Após o reset, PC = 0. O conteúdo dos registradores e da DMEM não é definido pela ISA: o programa deve inicializar o que usar, em especial o sp.
- Não há instrução de halt (ver 2.7).

2.10 Microarquitetura de referência (monociclo)

Caminho de dados:

- PC (registrador de 20 bits) e somador PC + 1. A IMEM é lida combinacionalmente com PC[11:0].
- A instrução é separada em campos: funct [3:0], rs2 [7:4], rs1 [11:8], rd [15:12], opcode [19:16].
- Banco de registradores com 2 leituras (Q1 de rs1, Q2 de rs2) e 1 escrita (D em rd, habilitada por w, na borda de subida do clock). O x0 não existe como registrador: a leitura devolve a constante 0.
- O endereço rs2 do banco vem de um multiplexador: bits [7:4] no formato R, bits [15:12] em S e B (sinal SFormat).
- ALU de 20 bits com entradas A = Q1 e B = Q2 ou imediato (sinal ALUSrc). Saídas: resultado R e o sinal Zero (R = 0).
- Código da ALU (3 bits): 000 ADD, 001 SUB, 010 AND, 011 OR, 100 XOR, 101 SLT, 110 SLL, 111 SRL. No tipo R, o código vem de funct[2:0] (sinal RType); nas demais, vem do campo ALUOp do controle.
- Multiplicador de 20 bits fora da ALU; seu resultado substitui o da ALU no write-back quando RType = 1 e funct[3] = 1.
- DMEM com endereço R[11:0], dado de entrada Q2 e escrita controlada por MemWrite.
- Write-back (ResultSrc): 00 = resultado da ALU (ou do mul), 01 = dado da DMEM, 10 = PC + 1.
- Próximo PC: PC + 1; PC + 1 + imediato (PCTarget); ou R da ALU (jalr). A seleção usa Taken OR Jump (bit 0) e Jalr (bit 1), onde Taken = (BranchEq AND Zero) OR (BranchNe AND NOT Zero) OR (BranchLt AND R[0]).
- Nos desvios, beq e bne fazem a ALU subtrair (ALUOp = 001) e usam o Zero; o blt faz a ALU executar slt (ALUOp = 101) e usa R[0].

Unidade de controle: uma ROM de 16 posições por 17 bits, endereçada pelo opcode. Campos da palavra: bit 16 RegWrite; bit 15 ALUSrc; bits 14-12 ALUOp; bit 11 MemWrite; bits 10-9 ResultSrc; bits 8-7 ImmSrc; bit 6 BranchEq; bit 5 BranchNe; bit 4 BranchLt; bit 3 Jump; bit 2 Jalr; bit 1 SFormat; bit 0 RType.

Opcode	Instrução	Valor da ROM (hex)
0	tipo R	10001
1	addi	18000
2	andi	1A180
3	ori	1B180
4	lw	18200
5	sw	08802
6	beq	01042
7	bne	01022
8	blt	05012
9	jal	10488
A	jalr	18404
B a F	reservados	00000

2.11 Constantes de 20 bits

Não há instrução de carga de constante larga. Constantes maiores que 8 bits são montadas com addi, sll e ori. Exemplo para 0x12345:

```
li   t0, 1          # t0 = 0x00001
li   t1, 8          # quantidade de deslocamento
sll  t0, t0, t1     # t0 = 0x00100
ori  t0, t0, 0x23   # t0 = 0x00123
sll  t0, t0, t1     # t0 = 0x12300
ori  t0, t0, 0x45   # t0 = 0x12345
```

2.12 Ferramentas e validação

- Montador (montador.py): converte assembly em hexadecimal (imagem da ROM do Logisim), binário (20 bits por linha) e listagem. Aceita rótulos, nomes de registradores (zero, ra, sp, gp, a0 a a3, t0 a t3, s0 a s3, ou x0 a x15) e as pseudoinstruções da seção 1.6.
- Programas: teste_todas.asm (todas as instruções, com desvios tomados e não tomados), fatorial.asm (função recursiva) e matriz.asm (multiplicação de matrizes 3×3), com os equivalentes em C.

Parte 3: Registradores e modelo de programação

3.1 Lista de registradores

Registrador	Nome	Função	Tipo	Quem preserva
x0	zero	constante 0 (escrita ignorada)	fixo	não se aplica
x1	ra	endereço de retorno	temporário	quem chama (caller)
x2	sp	ponteiro da pilha	salvo	quem é chamado (callee)
x3	gp	ponteiro global	fixo (não alterar)	não se aplica
x4	a0	argumento 1 e valor de retorno	temporário	quem chama (caller)
x5	a1	argumento 2 (e segundo valor de retorno)	temporário	quem chama (caller)
x6	a2	argumento 3	temporário	quem chama (caller)
x7	a3	argumento 4	temporário	quem chama (caller)
x8 a x11	t0 a t3	temporários	temporário	quem chama (caller)
x12 a x15	s0 a s3	valores que devem sobreviver a chamadas	salvo	quem é chamado (callee)

- Temporário (caller-saved): a função chamada pode alterar o registrador. Se quem chama precisa do valor depois da chamada, ele mesmo deve guardá-lo antes.
- Salvo (callee-saved): a função chamada deve devolver o registrador com o mesmo valor que tinha na entrada. Se precisar usá-lo, salva-o na pilha no início e restaura antes de retornar.

3.2 Convenção de chamada de função

- Argumentos: nos registradores a0 a a3, na ordem. Mais de 4 argumentos vão na pilha.
- Valor de retorno: em a0 (e em a1 se houver um segundo valor).
- Chamada: jal ra, função (grava o endereço de retorno em ra).
- Retorno: ret, isto é, jalr zero, ra, 0.
- Uma função que chama outra (não folha) deve salvar o ra na pilha, pois o jal sobrescreve o ra. Uma função folha pode usar ra sem salvar.
- Os registradores s0 a s3 e sp têm o mesmo valor na saída da função que tinham na entrada.

3.3 Pilha

- Cresce em direção a endereços menores. O sp aponta para a última palavra ocupada.
- Cada posição da pilha é uma palavra de 20 bits; não há alinhamento.
- Para reservar n palavras: addi sp, sp, -n. Para liberar: addi sp, sp, n.

- O hardware não inicializa o sp. Os programas de teste fazem ori sp, zero, 255 (topo da pilha na posição 255, longe dos dados nas posições baixas).

3.4 Exemplo: fatorial recursivo

```
fat:                        # a0 = n, retorna a0 = n!
    addi sp, sp, -2         # reserva 2 palavras
    sw   ra, 1(sp)          # salva o endereço de retorno
    sw   a0, 0(sp)          # salva n
    li   t0, 1
    blt  t0, a0, recur      # se 1 < n, chamada recursiva
    li   a0, 1              # caso base
    addi sp, sp, 2
    ret
recur:
    addi a0, a0, -1
    jal  ra, fat            # a0 = fat(n-1)
    lw   t1, 0(sp)          # recupera n
    lw   ra, 1(sp)          # recupera o retorno
    addi sp, sp, 2          # libera o quadro
    mul  a0, a0, t1         # n * fat(n-1)
    ret
```

O quadro da função guarda ra e n. O registrador t1, temporário, é lido da pilha depois da chamada, porque o valor de a0 e dos temporários não sobrevive a um jal.